



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Cruz Cano Omar

N° de Cuenta: 318068932

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027 - 1

FECHA DE ENTREGA LÍMITE: 23 de septiembre de 2026

CALIFICACIÓN: _____

Terminar el Rover con:

- Cabina.
- Base.
- Brazo con pinza con teclado se puede girar cada articulación.
- 6 llantas, se pueden girar cada una de las 6 llantas por separado.

Como se mencionó en el ejercicio, para la implementación de las 4 llantas traseras utilizamos una base en forma de cubo escalado en **X** para ser un prisma rectangular, luego un cilindro rotado 40 grados para poder movernos sobre **Y** como revisamos en clase, luego un segundo cilindro rotado 90 grados en **X** y escalado en **Y** para implementar el cilindro soporte, y por último otro cilindro con menor escala en **Y** y mayor escala en **X** y **Z** para la implementación de la rueda la cual tiene un color muy cercano a negro en todos los casos; todo esto utilizando el **modelaux2** para poder conectar correctamente todas las llantas a todo el rover.

```
// Base para llanta 1
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.3f, 4.4f));
color = glm::vec3(0.5f, 0.5f, 0.5f);
modelaux2 = model;
model = glm::scale(model, glm::vec3(1.5f, 0.75f, 0.75f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
```

Para implementar las otras 3 llantas solo fue necesario repetir código y modificar en los translates y rotates algún signo en donde fuera necesario, además cada soporte de llanta está pintado de un color diferente para lograr visualizarlos todos desde cualquier ángulo a ausencia de sombreado.

Todas las llantas giran de manera independiente utilizando **E, R, T, Y, U, I**; hicimos algunas modificaciones tanto en el **Window.h** como en **.cpp** para asignar cada tecla de nuestro teclado a una llanta diferente. Es importante inicializar las variables en 0.0f para no obtener resultados extraños

```
GLfloat getllanta3() { return llanta3; }
GLfloat getllanta4() { return llanta4; }
GLfloat getllanta5() { return llanta5; }
GLfloat getllanta6() { return llanta6; }

~Window();
private:
    GLFWwindow *mainWindow;
    GLint width, height;
    GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3;
    GLfloat llanta1, llanta2, llanta3, llanta4, llanta5, llanta6;
```

```

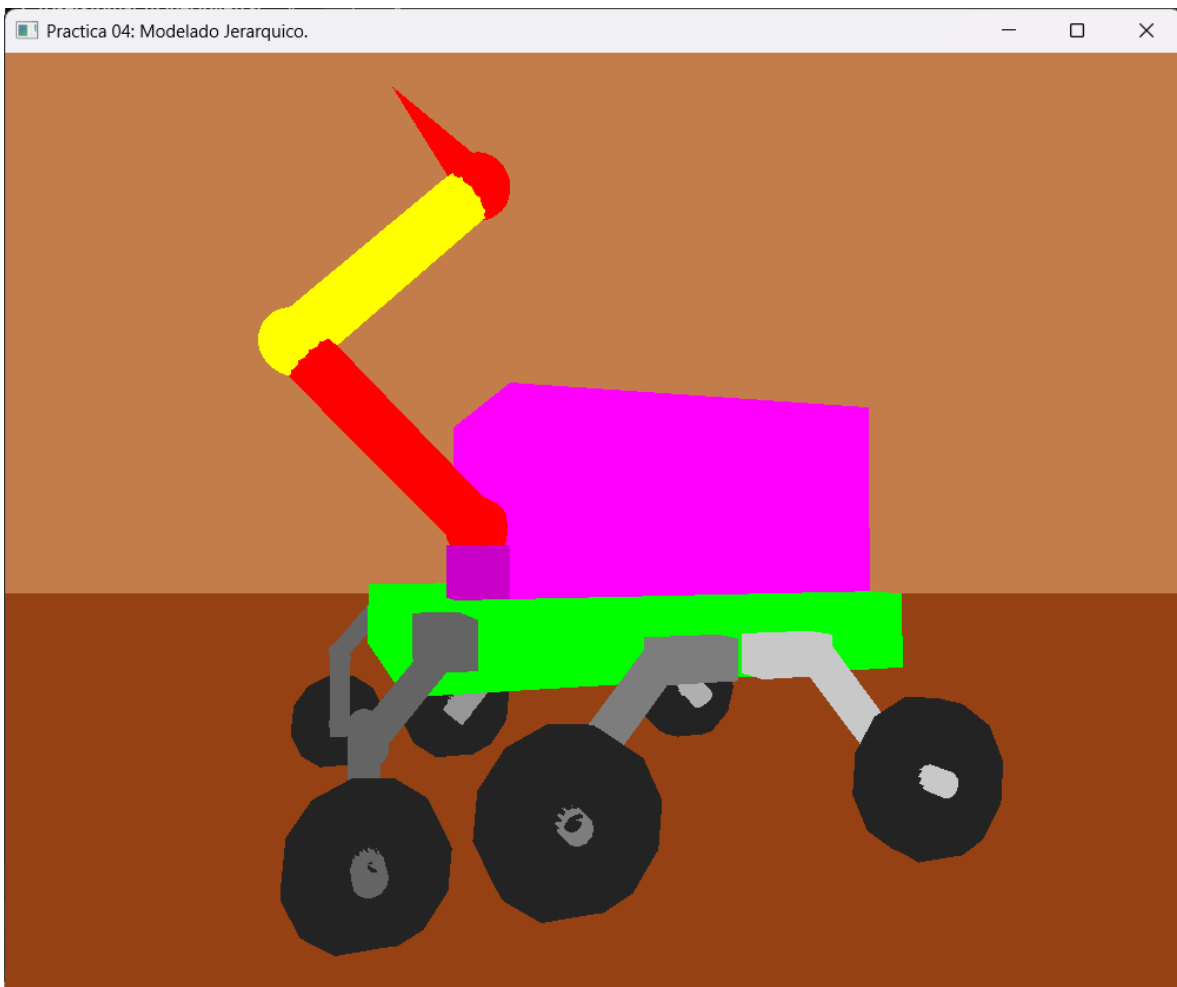
if (key == GLFW_KEY_E)
{
    theWindow->llanta1 += 10.0;
}
if (key == GLFW_KEY_R)
{
    theWindow->llanta2 += 10.0;
}

```

```

// LLanta 1
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, 0.4f, 0.0f));
color = glm::vec3(0.15f, 0.15f, 0.15f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del
model = glm::rotate(model, mainWindow.getllanta5() * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux2 = model;
model = glm::scale(model, glm::vec3(1.5f, 0.65f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMeshGeometry();

```



Crear Jerárquicamente una sonda espacial

Para la sonda empecé con un cilindro con una resolución de 8 como base, ya que buscaba que dicha base tuviera forma de prisma octogonal, dicha base está escalada rotada en **Z** 90 grados y escalada con un **Y** más elevado para hacerla más larga, utilizando modelado jerárquico implementé un cubo en lo que será la parte trasera de la sonda y podemos ver 3 antenas conectadas a dicho cubo, dos de ellas rotadas 45 grados respecto a **X** y una de ellas más corta con una esfera roja en la punta.

```
// Base cilíndrica
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 05.0f, -10.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 22.5f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, mainWindow.getsonda1() * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 2.0f, 1.5f));
color = glm::vec3(0.65f, 0.65f, 0.65f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMeshGeometry();
```

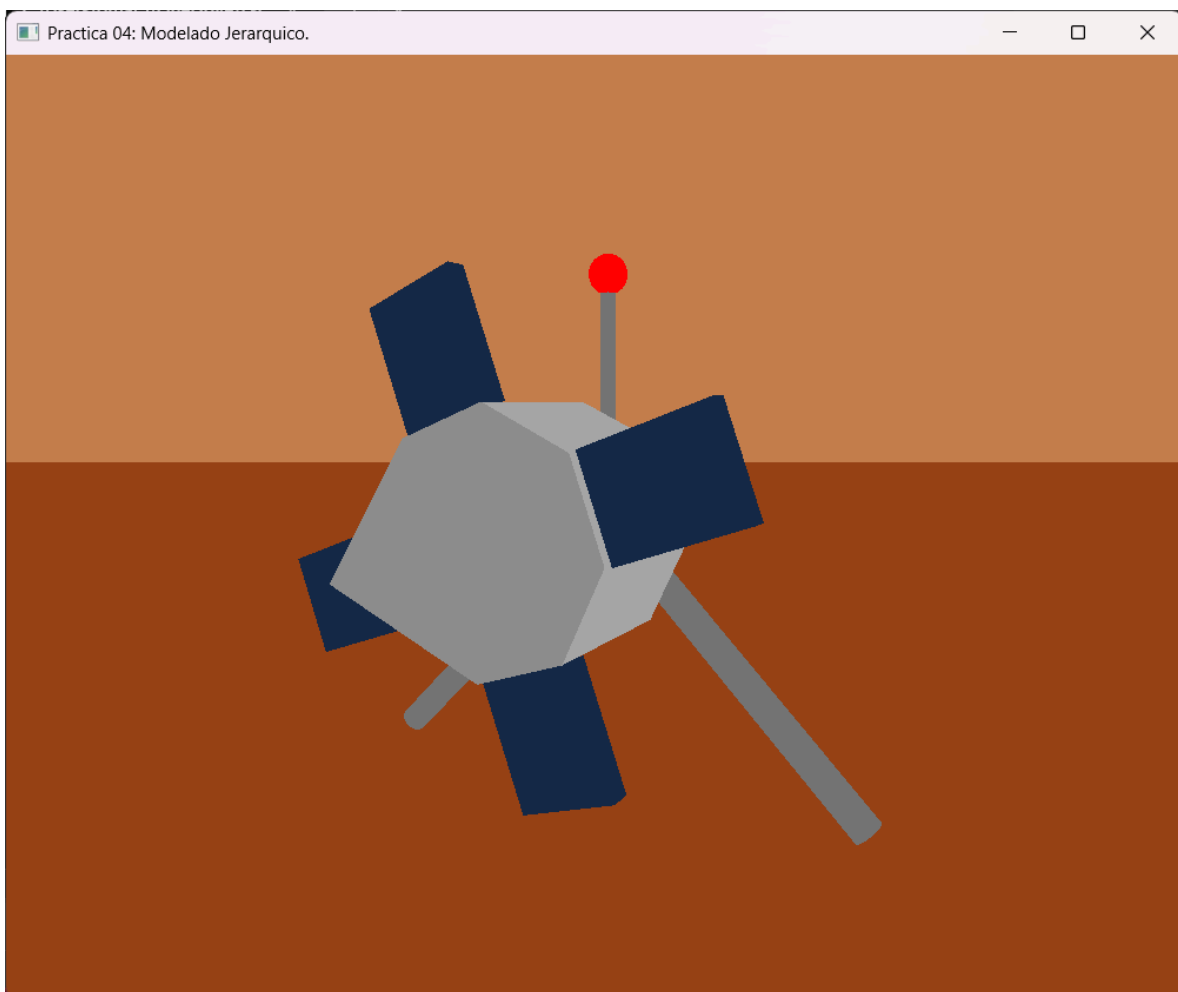
```
// Esfera roja
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, 1.2f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();
```

Gracias al modelado jerárquico, cargamos la información guardada anteriormente en el auxiliar para volver a la base y de ahí implementar a partir de cubos con una escala menor en el eje **X**, paneles solares de color azul los cuales estarán unidos directamente a la base octogonal. Después hicimos la implementación de un cono con la misma resolución de 8 como el cilindro para hacerla encajar y después de aplicar la rotación necesaria para lograr su estado final.

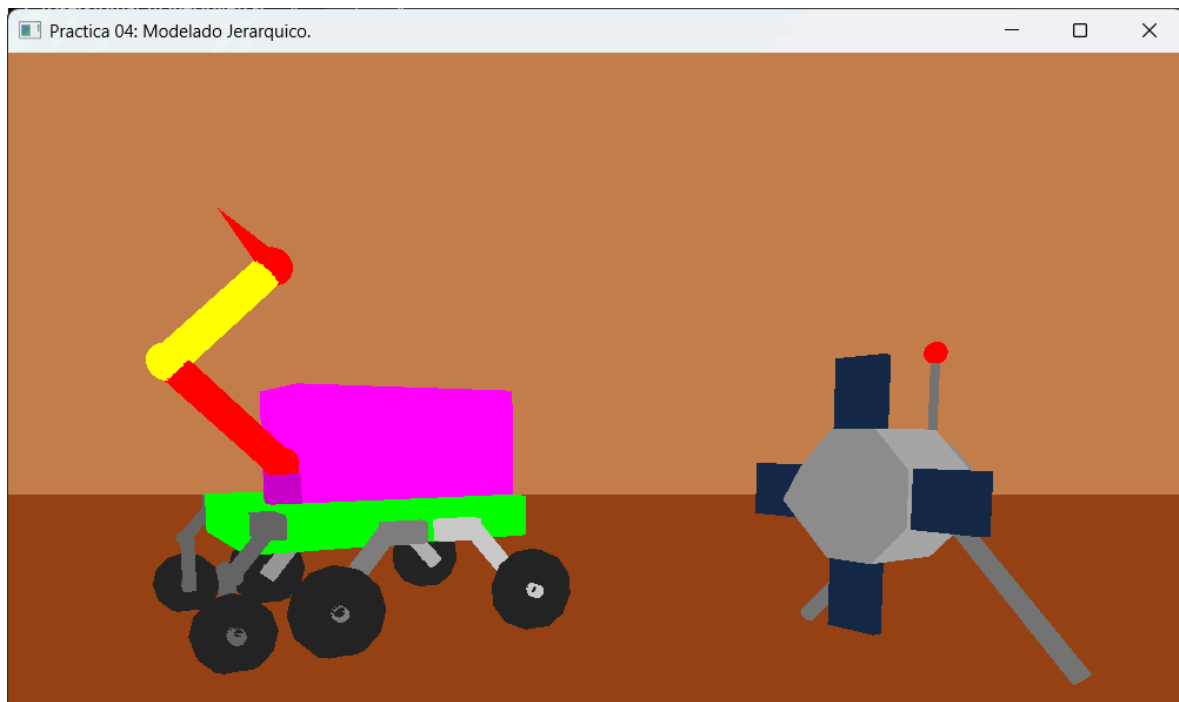
```
// Panel 2
model = modelaux;
model = glm::translate(model, glm::vec3(-0.75f, -2.14f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 1.5f, 1.2f));
color = glm::vec3(0.0941f, 0.1686f, 0.2863f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
```

```
// Cono
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, 0.0f, 0.0f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 22.5f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
color = glm::vec3(0.55f, 0.55f, 0.55f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
```

Para confirmar el correcto modelado de nuestra sonda podemos hacerla rotar utilizando **J** la cual rotará el modelo en el eje **Z**, la letra **K** girará en el mismo eje pero solamente la parte trasera de la sonda, y por último utilizando la letra **L** podremos hacer girar todo el modelo sobre el eje **Y**.



Se mantiene el fondo de color naranja y el suelo de color café asemejando al cielo y suelo de Marte, además de modificar la escala del cubo que constituye al suelo para poder tener un terreno mucho mayor. Por último, se hicieron las modificaciones pertinentes al **Window.cpp** para cambiar el nombre de la ventana.



Conclusión

Las actividades de este reporte permitieron afianzar el modelado jerárquico tridimensional mediante la construcción y articulación de modelos complejos como el rover y la sonda espacial. La mayor complejidad técnica radicó en manejar con precisión las matrices auxiliares para encadenar las transformaciones geométricas de elementos dependientes, además de aislar los grados de libertad para lograr que las seis llantas del vehículo giraran de manera independiente, lo cual también requirió adaptar meticulosamente los eventos de teclado desde los archivos **Window.h** y **.cpp** sin afectar la posición global del sistema.

La segunda actividad reforzó conocimientos revisados en sesiones anteriores ya que hicimos uso de la combinación de primitivas con resoluciones específicas (como los cilindros y conos de 8 caras) para ensamblar perfectamente las piezas del prisma octogonal de la sonda, lo que requirió definir el orden exacto de traslaciones, rotaciones y escalas, y aplicar adecuadamente la dependencia de nodos jerárquicos para permitir que distintas partes del modelo pudieran rotar de forma autónoma o en conjunto sobre los ejes Y y Z.